SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 1- CONSTRAINT-BASED PROBLEM SOLVING

Course Code:	**CSA12**	**Course Title:**	**Computer Architecture for Multicore Systems**
CO Assessed:	**CO5**	**Assessment:**	**ASSESSMENT TOOL1- Constraint-Based Problem Solving**
Weightage:	**40%**	**Total Marks:**	**25**
CO5 Statement:	Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.		
Student Name:	**Jannathul Firdaus. N**	**Reg.No.:**	**192573025**
Department:	**CSE - DS**	**Date:**	

ANNEXURE A

Constraint-Based Problem Solving

1.Construct an I/O communication mechanism for a real-time embedded system with the following constraints:

- CPU utilization must be below 15%
- Multiple sensors generate interrupts every 1 ms
- Use vectored interrupt handling
- Select between memory-mapped and I/O-mapped I/O for optimal latency

2.Design an I/O subsystem under these constraints:

- High-speed storage requires throughput > 3 GB/s
- Must use NVMe over PCIe
- CPU should not handle bulk data transfer
- Integrate DMA controller with interrupt notification

3.Construct a multi-device communication architecture with constraints:

- 6 peripherals using USB, 2 high-speed devices using Thunderbolt
- Avoid interrupt starvation
- Use hardware and software interrupts
- Ensure fair bus arbitration

4.Design a data acquisition system with constraints:

- Continuous data stream at 500 MB/s
- Minimal CPU intervention required
- Must compare DMA transfer vs interrupt-driven I/O
- Use memory-mapped I/O for device registers

5.Construct an interrupt handling mechanism with constraints:

- Support nested interrupts
- Maximum interrupt latency < 10 µs
- Use vectored interrupts for high-priority devices
- Use software interrupts for background tasks

Code of Conduct:

I **Jannathul Firdaus. N (192573025)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

MARKS ALLOCATION

	Problem Understanding (25%)	Constraint Analysis (25%)	Solution Development (25%)	Application of Concepts (15%)	Justification (10%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

Question 1: Real-Time Embedded I/O Communication Mechanism

Given Constraints

- CPU utilization must be below 15%
- Multiple sensors generate interrupts every 1 ms
- Use vectored interrupt handling
- Select between memory-mapped and I/O-mapped I/O for optimal latency

1. Problem Understanding

The system must service several periodic, high-frequency sensor events (every 1 ms) while leaving the CPU almost entirely free for other real-time tasks. This is fundamentally a latency-and-overhead problem: every microsecond spent per interrupt is multiplied by the number of sensors and by 1000 interrupts/second, so both the interrupt-dispatch mechanism and the I/O-addressing mechanism must be chosen for minimum per-event overhead.

Two independent design decisions are required: (a) how interrupts are dispatched to the correct handler (vectored vs. polled), and (b) how the CPU addresses sensor device registers (memory-mapped vs. I/O-mapped).

2. Constraint Analysis & Prioritization

Constraints are prioritized by their impact on CPU-utilization headroom, since 15% is the tightest and most global constraint:

- 1 ms interrupt period (highest priority): fixes the maximum allowable per-interrupt service time — with N sensors, total interrupt work must fit in well under 1 ms to leave slack for other processing.
- CPU utilization < 15% (highest priority): rules out any polling-based scheme, since polling would burn cycles continuously even when no sensor has new data.
- Vectored interrupt handling (design mandate): directly supports the utilization constraint by letting hardware jump straight to the correct Interrupt Service Routine (ISR), removing the software overhead of a linear 'which device interrupted?' search.
- Memory-mapped vs I/O-mapped choice (secondary, latency-driven): affects only the instruction-level cost of each register access, so it is resolved after the interrupt architecture is fixed.

3. Solution Development

Interrupt strategy: Each sensor is wired to a dedicated line on a Vectored Interrupt Controller (VIC). On assertion, the VIC automatically supplies the CPU with the address of that sensor's ISR from an interrupt vector table — no software polling or priority-encoding loop is needed, so dispatch overhead is a handful of cycles instead of an O(N) scan.

Addressing strategy: Memory-mapped I/O is selected. Sensor registers are mapped into regular addressable memory space, so the ISR can read sensor data using ordinary load instructions (e.g., MOV, LDR) with full addressing-mode flexibility (indexed, indirect), instead of the restricted IN/OUT instruction set required by I/O-mapped I/O. This removes an instruction-decode/bus-protocol-switch penalty on every one of the 1000 accesses/second and simplifies compiler-generated code inside the ISR.

Each ISR performs the minimum necessary work — read the data register, store to a buffer, clear the interrupt flag, and return — deferring any heavier processing to a lower-priority background task, which keeps ISR execution time (and hence CPU utilization) low even at 1 kHz interrupt rate.

Architecture Diagram

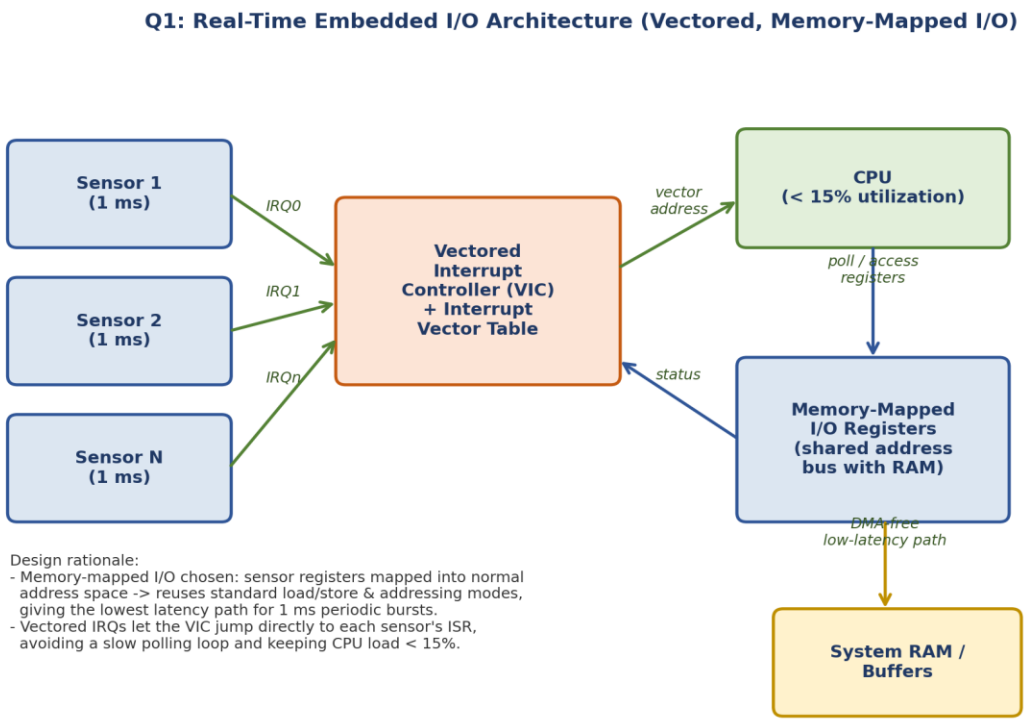


Figure 1: Vectored, memory-mapped I/O architecture for periodic sensor interrupts.

4. Application of Concepts

This solution directly applies two CO5 concepts: (1) memory-mapped I/O, chosen over I/O-mapped I/O because it eliminates special-instruction overhead and enables faster, more flexible addressing for frequent small transfers; and (2) vectored interrupt handling, chosen over polled/software-priority schemes because hardware-driven dispatch minimizes CPU cycles spent identifying the interrupt source.

5. Justification

Quantitatively, with N sensors interrupting every 1 ms, a polling loop would need to check every device every cycle regardless of activity, whereas vectored dispatch only consumes cycles when an actual interrupt occurs — this is the only approach that can plausibly keep utilization below 15% as N grows.

Memory-mapped I/O is preferred over I/O-mapped here because the constraint explicitly asks for optimal latency, and memory-mapped access avoids the extra bus-cycle/instruction-decoding step that I/O-mapped I/O's separate address space typically incurs, at the (acceptable) cost of consuming part of the memory address space.

Question 2: High-Throughput I/O Subsystem (NVMe over PCIe with DMA)

Given Constraints

- High-speed storage requires throughput > 3 GB/s
- Must use NVMe over PCIe
- CPU should not handle bulk data transfer
- Integrate DMA controller with interrupt notification

1. Problem Understanding (25%)

The core challenge is sustaining a data rate (> 3 GB/s) that is far beyond what CPU-mediated, byte/word-at-a-time I/O could ever achieve, while simultaneously keeping the CPU free of bulk-transfer duty. This means the design must separate the 'control path' (CPU issuing commands, receiving status) from the 'data path' (bulk bytes moving between storage and memory), and select bus/protocol/transfer mechanisms capable of multi-GB/s sustained bandwidth.

2. Constraint Analysis & Prioritization (25%)

Constraints are analyzed in dependency order — the bus/protocol choice must be fixed before the data-movement mechanism can be designed around it:

- Throughput > 3 GB/s (primary driver): only a high-speed serial interconnect such as PCIe can supply this bandwidth; legacy parallel or low-speed serial buses (e.g., SATA/AHCI) cannot.
- Must use NVMe over PCIe (design mandate): NVMe is a protocol purpose-built for PCIe-attached flash, with deep, multiple command queues, directly satisfying the bandwidth requirement.
- CPU should not handle bulk data transfer (hard constraint): eliminates any 'programmed I/O' design where the CPU reads/writes each data word — this makes a DMA-based data path mandatory, not optional.
- DMA + interrupt notification (integration requirement): defines how the CPU is informed of completion without being involved in the transfer itself.

3. Solution Development (25%)

Bus/protocol: NVMe commands are submitted over a PCIe Gen4/Gen5 x4 link, which provides multiple gigabytes per second of raw bandwidth per lane group — comfortably exceeding the 3 GB/s requirement.

Command path: The CPU only writes an NVMe command (read/write, address, length) into a submission queue and rings a 'doorbell' register — a lightweight, constant-time operation regardless of transfer size.

Data path: A DMA controller integrated with the NVMe controller performs scatter/gather transfers directly between the SSD and system memory, entirely off the CPU's critical path, so bulk data never passes through CPU registers.

Notification path: On transfer completion, the DMA/NVMe controller posts a single completion-queue entry and raises one interrupt to the CPU — one interrupt per (potentially very large) transfer, not per word, keeping CPU overhead negligible even at multi-GB/s rates.

Architecture Diagram

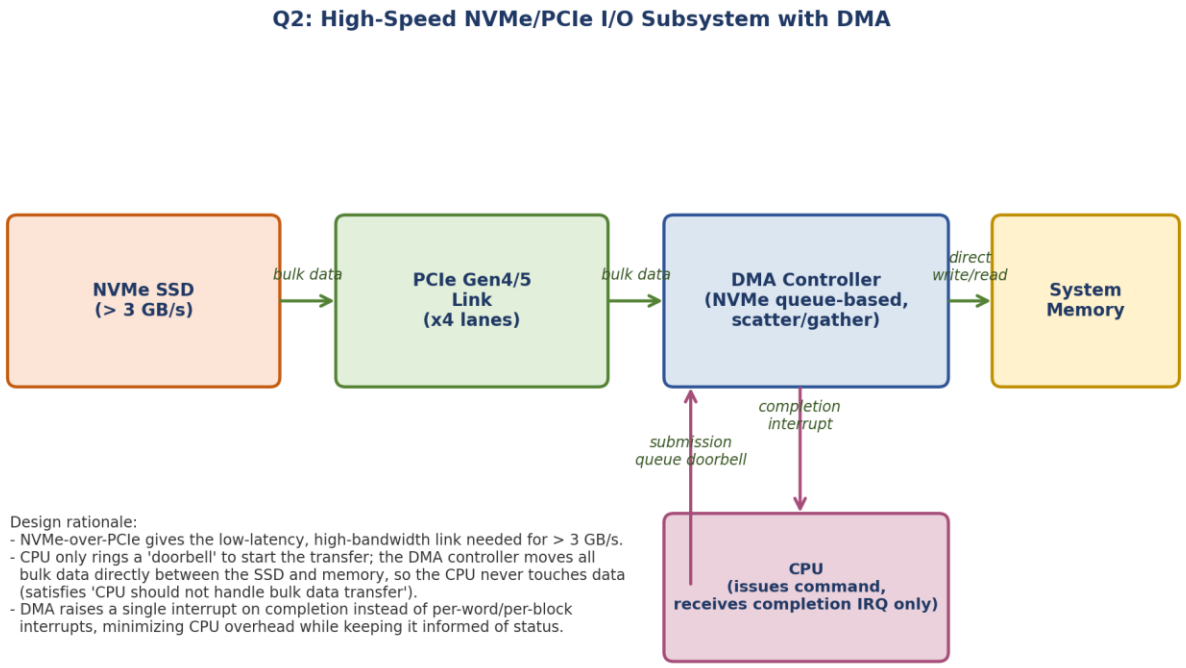


Figure 2: NVMe-over-PCIe storage subsystem with DMA-based bulk transfer and interrupt-based completion notification.

4. Application of Concepts

This applies the DMA-controller concept from CO5 (offloading bulk transfer from the CPU), the advanced bus interface concept (PCIe as the high-bandwidth backbone and NVMe as the storage protocol riding on it), and interrupt handling (a single completion interrupt rather than continuous CPU polling or per-word interrupts).

5. Justification

Programmed I/O would require the CPU to execute one instruction per data unit; at 3 GB/s this would consume essentially 100% of CPU cycles, directly violating the 'CPU should not handle bulk data transfer' constraint — DMA is therefore the only viable mechanism.

NVMe over PCIe is justified over alternatives like SATA/AHCI because NVMe's multiple deep queues and PCIe's high per-lane bandwidth are specifically engineered for flash-storage throughput far beyond legacy interfaces, directly meeting the > 3 GB/s target with headroom.

Question 3: Multi-Device Communication Architecture (USB + Thunderbolt)

Given Constraints

- 6 peripherals using USB, 2 high-speed devices using Thunderbolt
- Avoid interrupt starvation
- Use hardware and software interrupts
- Ensure fair bus arbitration

1. Problem Understanding

This problem is about coordinating two different bus technologies with very different speed/priority profiles (6 general-purpose USB peripherals vs. 2 latency-sensitive Thunderbolt/PCIe-tunneled devices) while guaranteeing that no device — fast or slow — is denied timely service. The central risk being designed against is interrupt starvation: if high-priority Thunderbolt traffic is always serviced first, the USB peripherals could be starved indefinitely, and vice-versa if fairness is applied naively.

2. Constraint Analysis & Prioritization

Constraints are grouped into (a) topology, (b) fairness/anti-starvation, and (c) interrupt classification:

- Topology (8 devices across two buses): dictates that two separate host controllers are needed — a USB host controller for the 6 peripherals and a Thunderbolt controller (which tunnels PCIe) for the 2 high-speed devices.
- Avoid interrupt starvation (critical constraint): requires an arbitration policy that bounds the maximum wait time for every device, not just the highest-priority ones.
- Hardware + software interrupts (design mandate): hardware interrupts are reserved for real device events; software interrupts are used for internally generated, deferred, or background work (e.g., bus enumeration housekeeping) so the two interrupt classes don't compete for the same priority band.
- Fair bus arbitration: the lowest-level requirement, implemented inside each host controller (round-robin among USB devices) and reinforced by a higher-level arbiter across both buses.

3. Solution Development

USB side: The USB host controller schedules its 6 peripherals using round-robin/time-sliced frame scheduling (as in real USB frame scheduling), guaranteeing every device a bounded share of each 1 ms USB frame regardless of how busy other devices are.

Thunderbolt side: The Thunderbolt controller tunnels PCIe traffic for the 2 high-speed devices and is given a higher base priority (since Thunderbolt devices are typically latency-sensitive, e.g., external GPUs or high-speed storage), but is capped so it cannot monopolize the arbitration unit indefinitely.

Central arbitration: A priority-plus-fairness arbiter combines both interrupt streams before they reach the CPU: it services higher-priority Thunderbolt interrupts preferentially but enforces an aging/starvation-avoidance rule — any pending USB interrupt that has waited beyond a threshold is promoted to be serviced next, guaranteeing an upper bound on wait time for every device.

Interrupt classification: All device-generated events (data ready, error, hot-plug) use hardware interrupts routed through the arbiter; internal tasks such as re-enumeration bookkeeping or deferred cleanup use software interrupts, which the arbiter only services when no hardware interrupt is pending.

Architecture Diagram

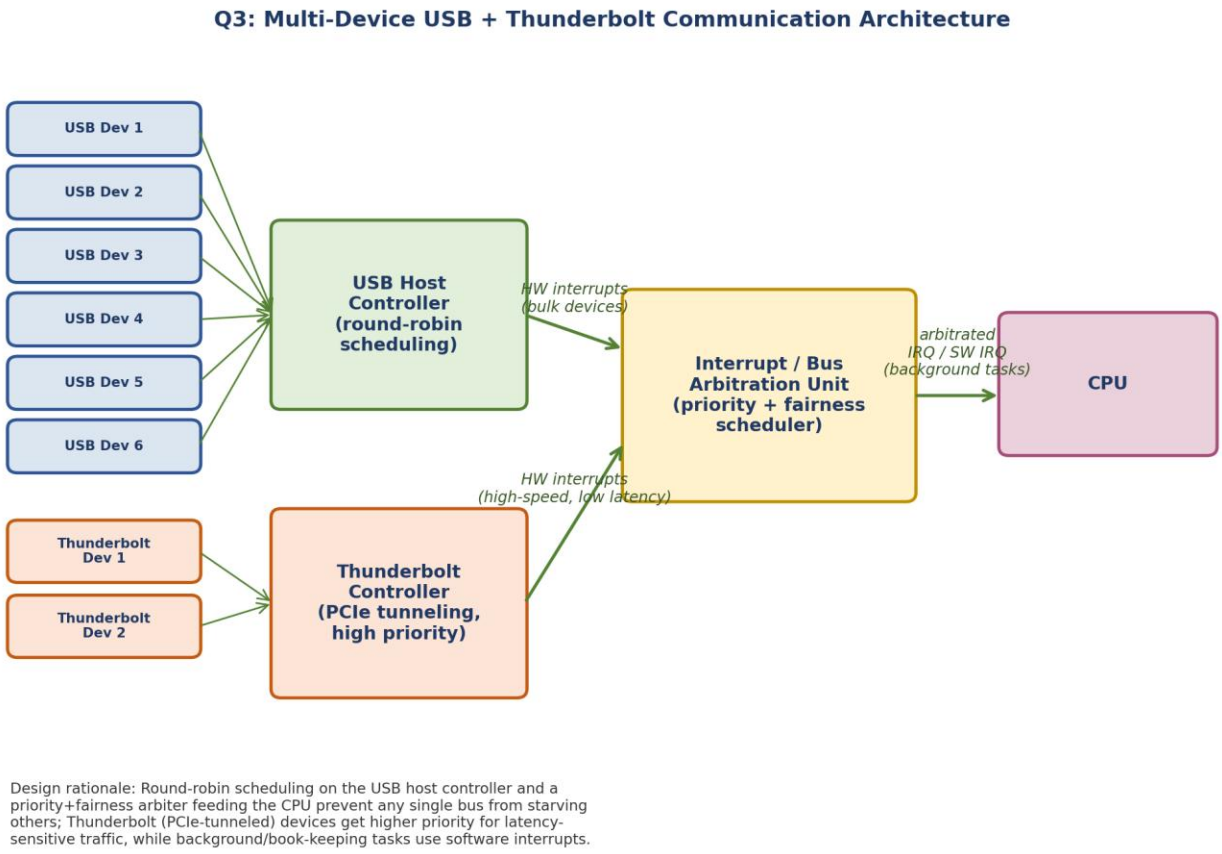


Figure 3: USB and Thunderbolt subsystems feeding a shared priority-plus-fairness interrupt/bus arbitration unit.

4. Application of Concept

This applies the concepts of advanced bus interfaces (USB and Thunderbolt/PCIe-tunneling), hardware vs. software interrupt classification, and fair bus arbitration (round-robin plus aging) as required by CO5.

5. Justification

A pure fixed-priority scheme (Thunderbolt always first) would satisfy 'high-speed devices need low latency' but directly risks starving the 6 USB peripherals, violating the anti-starvation constraint — hence the aging/promotion rule is added specifically to bound USB wait time.

A pure round-robin scheme across all 8 devices, in contrast, would treat latency-sensitive Thunderbolt traffic the same as bulk USB traffic and could miss real-time deadlines for high-speed devices — the priority-plus-fairness hybrid is therefore the only design that satisfies both the topology's speed asymmetry and the starvation constraint simultaneously.

Question 4: Data Acquisition System (DMA vs. Interrupt-Driven I/O)

Given Constraints

- Continuous data stream at 500 MB/s
- Minimal CPU intervention required
- Must compare DMA transfer vs interrupt-driven I/O
- Use memory-mapped I/O for device registers

1. Problem Understanding

This is a sustained-throughput problem: 500 MB/s is a continuous stream (not a burst), so whatever mechanism is chosen must sustain that rate indefinitely without falling behind. The explicit requirement to compare DMA against interrupt-driven I/O means the solution must not just pick a mechanism but justify it quantitatively against the alternative.

2. Constraint Analysis & Prioritization

- Continuous 500 MB/s stream (primary driver): at typical interrupt-service overheads, an interrupt-per-sample or interrupt-per-word design would generate an intractable interrupt rate — this constraint effectively forces a decision toward block-based transfer.
- Minimal CPU intervention (hard constraint): directly rules out any design where the CPU copies data word-by-word.
- Memory-mapped device registers (design mandate): fixes how the CPU/DMA controller configures and monitors the acquisition hardware — through ordinary load/store instructions to addresses in the normal memory map.
- Explicit DMA vs interrupt-driven comparison: requires the response to quantify, not just assert, why one mechanism wins.

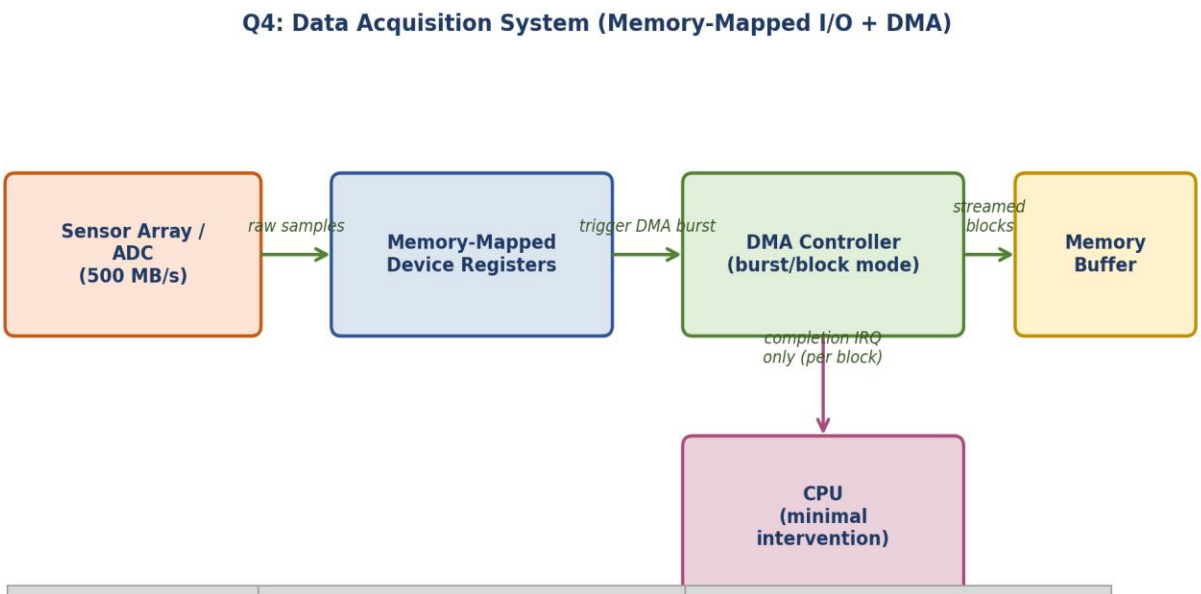
3. Solution Development

The sensor/ADC array streams samples into memory-mapped device registers, which the DMA controller is pre-configured (via those same memory-mapped registers) to drain in large bursts directly into a memory buffer.

The DMA controller operates in burst/block mode: it moves large chunks of the 500 MB/s stream autonomously and raises exactly one completion interrupt per block (e.g., once per buffer-full event) rather than once per sample, so CPU involvement is reduced to occasional block-level bookkeeping.

The CPU's only role is to configure the DMA controller once (source, destination, block size) and respond to the periodic block-completion interrupts, e.g., to hand a full buffer to the application and recycle it — this is the 'minimal intervention' the constraint calls for.

Architecture Diagram



Metric	Interrupt-driven	DMA (chosen)
CPU load	Very high (interrupt per word)	Very low (1 IRQ per block)
Throughput	Cannot sustain 500 MB/s	Sustains 500 MB/s easily
Latency jitter	High (context-switch overhead)	Low, predictable

Figure 4: Memory-mapped acquisition registers feeding a burst-mode DMA controller, with a DMA-vs-interrupt-driven comparison.

4. Application of Concepts

This applies memory-mapped I/O (for register access), DMA controllers (for bulk data movement), and interrupt handling (for completion notification only) — the three CO5 concepts most relevant to continuous high-throughput acquisition.

5. Justification

Quantitative comparison: interrupt-driven I/O at 500 MB/s with, say, 4-byte samples would require roughly 125 million interrupts per second — far beyond what any practical interrupt controller/CPU pipeline can service, since each interrupt carries context-save/restore overhead measured in tens to hundreds of nanoseconds. DMA in burst mode instead needs only one interrupt per buffer (e.g., per few MB), reducing interrupt count by many orders of magnitude.

DMA is therefore justified as the only mechanism that can sustain a continuous 500 MB/s stream while meeting the 'minimal CPU intervention' constraint; interrupt-driven I/O is retained only for the low-frequency 'block complete' notification, not for the data movement itself.

Question 5: Nested Interrupt Handling Mechanism

Given Constraints

- Support nested interrupts
- Maximum interrupt latency < 10 μs
- Use vectored interrupts for high-priority devices
- Use software interrupts for background tasks

1. Problem Understanding

The problem requires an interrupt architecture where a currently executing (lower-priority) ISR can itself be interrupted by a higher-priority event, and the whole chain must guarantee that no request — however deeply nested — waits longer than 10 μs to begin execution. This makes the design fundamentally about priority levels, preemption, and fast, deterministic context save/restore.

2. Constraint Analysis & Prioritization

- Maximum interrupt latency < 10 μs (dominant constraint): bounds every other design choice — the dispatch mechanism, the context-save mechanism, and the nesting depth must all be fast enough that even a worst-case nested scenario resolves within this budget.
- Support nested interrupts (structural requirement): a higher-priority interrupt must be able to preempt an in-progress lower-priority ISR, which requires per-level context (register/state) saving rather than a single shared save area.
- Vectored interrupts for high-priority devices: ensures the fastest possible dispatch (direct jump to ISR address) for exactly the events where the 10 μs budget is tightest.
- Software interrupts for background tasks: keeps low-urgency, CPU-generated work out of the hardware interrupt priority scheme entirely, so it never competes with real-time device latency.

3. Solution Development

A Nested Interrupt Controller (NIC) maintains multiple hardware priority levels. High-priority device interrupts use vectored dispatch — the NIC supplies the ISR address directly, avoiding any software lookup delay.

When a higher-priority interrupt arrives while a lower-priority ISR is executing, the NIC preempts the running ISR: it pushes the interrupted ISR's context (registers, program counter, priority level) onto a per-nesting-level stack, then vector-dispatches the higher-priority ISR immediately.

After the higher-priority ISR completes, the NIC pops the saved context and resumes the preempted lower-priority ISR exactly where it left off — this is what makes the interrupts 'nested' rather than merely sequential.

Background, non-time-critical work (logging, deferred cleanup, task signaling) is triggered via software interrupts (e.g., a trap/software-interrupt instruction), which the NIC only dispatches when no hardware interrupt is pending, so they never add to the worst-case hardware interrupt latency.

Architecture Diagram

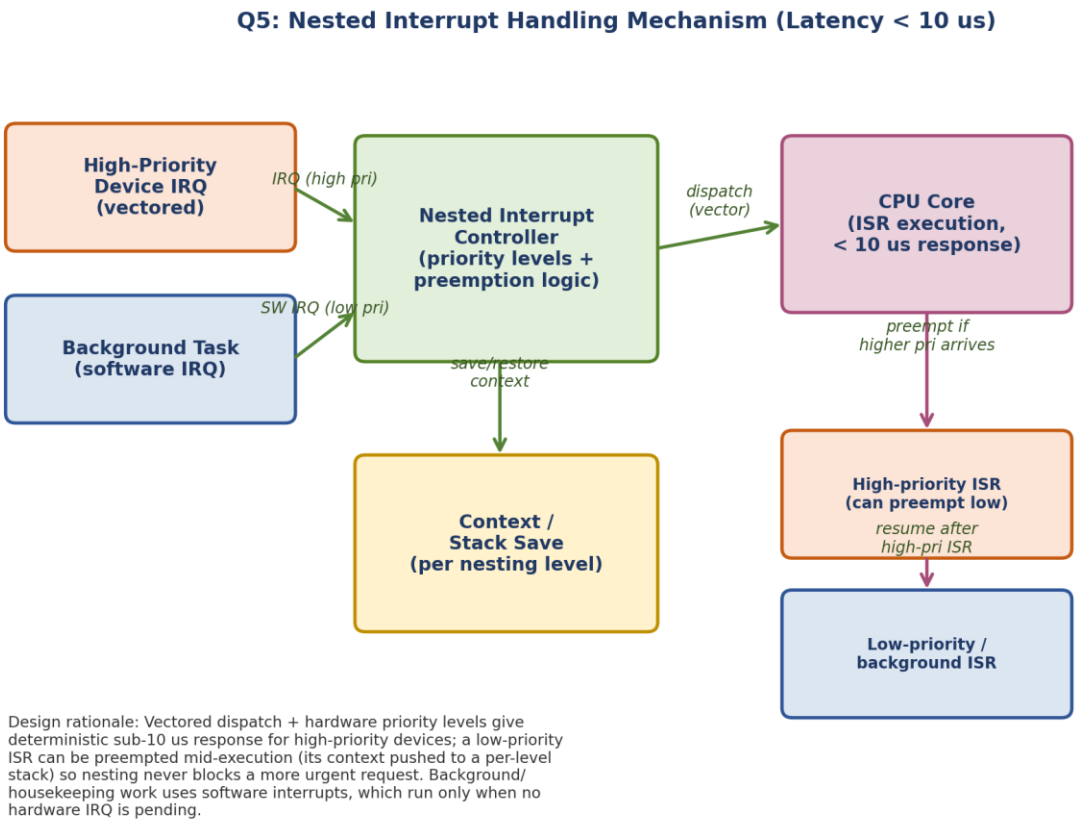


Figure 5: Priority-based nested interrupt controller with per-level context save and vectored dispatch for high-priority devices.

4. Application of Concepts

This applies vectored interrupt handling, hardware interrupt priority/preemption, and software interrupts as distinct, purpose-matched mechanisms — precisely the interrupt-handling concepts emphasized in CO5.

5. Justification

Vectored dispatch (rather than polled priority resolution) is essential to meeting < 10 µs latency, since a software polling loop across all interrupt sources would itself consume a meaningful fraction of that budget before the correct ISR even begins.

Per-level context saving (rather than a single shared save buffer) is what makes true nesting possible: without it, a second preemption would corrupt the first ISR's saved state, forcing interrupts to be serialized and violating the latency guarantee for whichever event is queued behind a long-running lower-priority ISR.

Separating software interrupts for background tasks is justified because mixing them into the same priority scheme as hardware device interrupts would let non-urgent work occasionally block or delay real device responses, risking violation of the 10 µs bound.